Evaluate AI Systems

Application-specific criteria, model selection, benchmarks, and repeatable evaluation pipelines

Select and improve the whole system against your users' tasks. Public rankings help shortlist models; private, representative evaluations determine whether a configuration is useful enough to deploy.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 159–210 / PDF p. 183. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 Evaluation Criteria

Source: book pp. 160–179 / PDF p. 184

Evaluation-driven development and domain capability

Define what success and failure mean before optimizing. Evaluation-driven development focuses investment on applications whose outcomes can be measured. Tie offline quality to business results, recognizing that correlation does not establish causation; product experiments help isolate impact.

Domain-specific capability includes code, mathematics, legal reasoning, and other specialized tasks. Use executable checks where possible and carefully designed expert labels elsewhere. Multiple-choice accuracy can be useful but depends on answer format, partial-credit rules, option ordering, and whether the benchmark resembles actual use. Correct code may still be too slow or expensive.

Generation capability: fluency, coherence, and relevance

Fluency concerns natural language quality; coherence concerns logical organization across the response. A fluent response can still be irrelevant, incorrect, or unhelpful. Translation faithfulness and summarization relevance require checking meaning against the source rather than rewarding polished wording.

Measure attributes separately before deciding how to combine them. An overall average can hide a critical failure: strong writing quality does not compensate for fabricated technical evidence or a privacy violation.

Local versus global factual consistency

Local consistency/faithfulness: is a claim supported by the supplied context? **Global factuality:** is it correct against trustworthy external knowledge? A source-grounded answer can faithfully repeat a false source, while a true statement may be unsupported by the provided source.

Decompose responses into checkable claims, retrieve trustworthy evidence, and assess support. SAFE-style workflows illustrate claim-level search and verification. Natural-language inference classifies a premise/hypothesis pair as entailment, contradiction, or neutral. Neutral means insufficient evidence, not automatically false. Absence of found evidence is not evidence of absence. Evaluate retrieval authority as well as relevance.

Safety evaluation

Define a harm taxonomy relevant to the product: privacy exposure, harmful content, discriminatory behavior, unsafe instructions, or misinformation. Use moderation/scoring systems and adversarial tests appropriate to the population, language, and domain.

Measure both unsafe compliance and unnecessary refusal. Refusing legitimate requests can make a system unusable; permissive behavior can expose users to harm. Public toxicity tests are useful baselines, not a substitute for application-specific cases. Record severity and affected groups instead of collapsing every failure into a single equally weighted count.

Instruction-following capability and role-playing

Separate "cannot do the task" from "did not follow the instruction." Test verifiable constraints: required/forbidden terms, word or paragraph limits, JSON fields, response structure, and combinations of constraints. IFEval-style tests use constraints that can be checked programmatically; other benchmarks use human or AI judgments.

Role-playing requires the correct voice, background knowledge, and boundaries on knowledge. A character should not reveal facts it could not know or future story spoilers. Evaluate persona consistency across turns and interactions, not only a single convincing introduction.

Cost and latency as first-class criteria

Track total response time, time to first visible token, per-token generation time, and tail percentiles. Multi-step pipelines can hide internal token generation, making user-perceived TTFT much higher than a model's own TTFT.

Estimate full request cost: generation, retrieval, embeddings, tools, scoring, retries, storage, and operations. Set hard requirements and desirable targets for the actual workload. Test with realistic input/output lengths and concurrency; advertised per-token prices alone do not describe cost per successful task.

02 Model Selection Workflow

Source: book pp. 179–181 / PDF p. 203

A practical selection loop

1. Define hard requirements, improvement targets, and permitted deployment options.
2. Shortlist using documentation, model cards, licenses, and relevant public evidence.
3. Evaluate candidates on private examples; estimate whether prompt, retrieval, or fine-tuning changes can close gaps.
4. Deploy cautiously, monitor live outcomes, and revise as evidence changes.

Hard versus soft depends on control. Unsupported modalities can be hard constraints; latency may be adjustable if you control serving. Do not spend weeks optimizing a model that fundamentally cannot meet the task.

03 Model Build Versus Buy

Source: book pp. 181–191 / PDF p. 205

Open weights, source, data, and licenses

Terminology is inconsistent: available weights do not necessarily imply open training data, unrestricted commercial use, or a reproducible training process. Read the actual license and service terms for fine-tuning, redistribution, output reuse, distillation, and competing services.

Model ownership and serving ownership are separate decisions. An open-weight model can be accessed through a managed API, while self-hosting requires compute, deployment, security, upgrades, and on-call expertise.

Seven decision axes

- **Privacy:** residency, retention, access, and whether inputs may be reused.
- **Lineage:** source-data rights and contractual/legal responsibilities.
- **Performance:** measured task quality and capacity, not marketing claims.
- **Functionality:** modalities, context, log probabilities, fine-tuning, tools, guardrails, and structure support.
- **Cost:** workload-dependent total ownership, engineering, and utilization.
- **Control/access:** rate limits, offline/edge requirements, version freezing, deprecation, and outages.
- **Support/transparency:** community, vendor help, debugging access, and change visibility.

Avoid simplistic hosting assumptions

Self-hosting is not automatically cheaper or more private; it transfers responsibility to your team. APIs are not automatically unsafe; evaluate actual agreements, deployment regions, and controls. Public training data is not trivial to audit at foundation-model scale.

Different providers serving the same nominal model can differ in templates, quantization, runtime, and limits. Switching endpoint providers therefore requires tests. Reassess the trade-off as prices and requirements change.

---

04 Navigate Public Benchmarks

Source: book pp. 191-200 / PDF p. 215

Benchmark selection and aggregation

Choose relevant capabilities rather than copying the largest leaderboard. Check what each benchmark measures, its scoring protocol, number and kind of examples, prompt settings, contamination risk, and uncertainty. Examples in the book include MMLU, HumanEval, GSM8K, ARC, HellaSwag, TruthfulQA, WinoGrande, BBH, and IFEval.

Highly correlated benchmarks can double-count the same ability; aggregation weights change model rankings. Saturated benchmarks provide little discrimination. Compare base and instruction variants under comparable prompts and sampling, and distinguish real capability shifts from formatting or evaluator artifacts.

Data contamination

Public benchmark items can enter web training corpora deliberately or accidentally. Exact/n-gram overlap with training data is direct but requires access and computation; unusually low perplexity is cheaper but weaker evidence. A suspicious score is a reason to investigate, not proof.

Private holdouts, protected evaluation services, refreshed examples, and clean-versus-full comparisons help. Performance gaps between clean and contaminated items can vary. A public benchmark helps reject weak candidates; your own protected task set is still needed to select the best fit.

---

05 Design Your Evaluation Pipeline

Source: book pp. 200-210 / PDF p. 224

Step 1: Evaluate all components and the task

Test retrievers, routers, generators, tool calls, and scorers individually to diagnose defects, then test end-to-end outcomes. A component improvement can worsen the whole system by adding latency or suppressing useful context.

Distinguish per-turn quality from per-task completion across a conversation. A clarifying question may look incomplete in one turn but be necessary to solve the task. Check downstream effects and final outcomes, including safety, cost, and human effort.

Step 2: Create guidelines and rubrics

Specify desired behaviors, prohibited behaviors, criteria, severity, score meanings, and concrete examples. Have humans apply the rubric and resolve ambiguity before using it to calibrate a model judge. A clear rubric is foundational to both training labels and evaluation labels.

Relate metrics to meaningful outcomes rather than optimizing proxies blindly. Session length can mean engagement or frustration depending on the product. Optimize business value without ignoring user welfare and risk.

Step 3: Define methods and evaluation data

Combine exact checks, similarity, judges, and experts according to failure mode. Curate real-use examples when allowed and include frequent tasks, edge cases, typos, out-of-scope requests, and known failures. Protect the evaluation set from model training and prompt overfitting.

Report slices by task, user group, source, or platform. **Simpson's paradox** can reverse an aggregate ranking when group proportions differ, so preserve comparable distributions. Track label disagreement and quality. Smaller score differences require more samples; sample requirements scale roughly with the inverse square of the effect size under common statistical assumptions.

Evaluate your evaluator and iterate

Measure human/judge agreement, reproducibility, variance, criterion correlations, cost, and coverage. Pin configurations and make changes explicit. Use uncertainty intervals and repeated runs when needed.

New production failures become regression cases. Update criteria when user needs change, but retain evaluation versions so historical comparisons remain interpretable. Keep a minimum viable evaluation suite early; grow it with evidence instead of postponing measurement until launch.

Revision Check

1. Can an answer be locally faithful but globally false?
2. Which requirements are hard constraints for your deployment?
3. What can a leaderboard average hide?

4. How do per-turn and per-task evaluations differ?
5. How would you detect an unreliable judge or a shift in evaluation data?

> A useful evaluation pipeline is a versioned product asset: it measures what matters, explains failures, and changes deliberately as the system and its users evolve.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.